Challenge (3 public points)

In this challenge, you will use the Brown–Doyle mammalian clock model to propose parameter changes that bring a reporter-perturbed clock back toward its wild-type behaviour (i.e. the "normal" period and relative peak timing of selected species).

Say the parameters of a wild-type animal are given by $\mathbf{p}_{\mathrm{WT}}$. Through genetic engineering, you attach a luminescent reporter to a protein in an animal so that you can measure this protein over time (e.g. to measure the effect of another gene on this protein). The issue, as mentioned on the first page, is that attaching that reporter can change the dynamics of the entire system before you even got to the actual experiment. In other words, the parameters are now $\mathbf{p}_{\mathrm{rep}}$.

The question is: how do you get the system back to normal? The easy answer is "remove the reporter", but that of course defeats the purpose of using it for measurements. Instead, you have to consider adding more perturbation using methods that further change the parameters:

$$\mathbf{p}_{\mathrm{WT}} \xrightarrow{\text{add reporter}} \mathbf{p}_{\mathrm{rep}} \xrightarrow{\text{further modifications } \Delta\mathbf{p}} \mathbf{p}_{\mathrm{edit}}$$

The hope is that, even though $\mathbf{p}_{\mathrm{WT}} \neq \mathbf{p}_{\mathrm{edit}}$, the proteins in the two animals behave the same. That means you now have an animal that behaves like the wild-type but with a protein tracker.

The question is: Given $\mathbf{p}_{\mathrm{rep}}$, how do you choose $\Delta\mathbf{p}$ so that $\mathbf{p}_{\mathrm{edit}} = \mathbf{p}_{\mathrm{rep}} + \Delta\mathbf{p}$ gives "normal behaviour" again? Furthermore, how do you do this using only possible modifications (since, due to the biological reality, you can modify only certain groups of DNA-associated parameters).

Setup. The starter files provide the Brown–Doyle ODE model, the target wild-type period and peak phases, and the reporter-perturbed parameter vector $\mathbf{p}_{\mathrm{rep}}$. The wild-type parameter vector itself is not supplied.

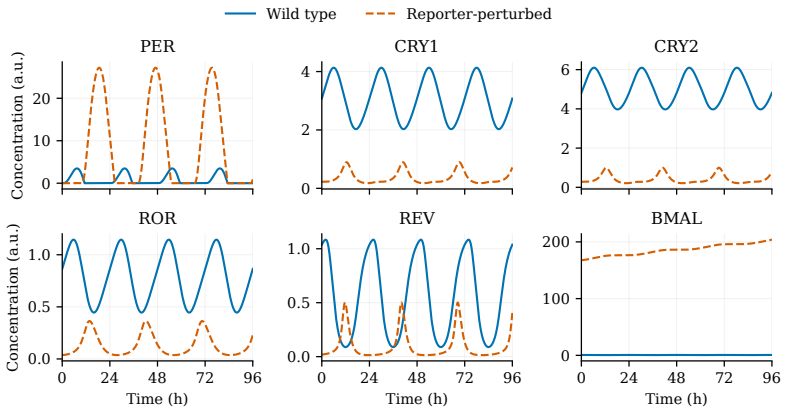


Figure 1: *Wild-type and reporter-perturbed protein concentrations for the six core clock genes in the Brown–Doyle model. The reporter changes both oscillations and concentration levels; reporter-perturbed BMAL continues to drift over this window.*

Your goal is to restore the wild-type behaviour by modifying DNA-associated parameters of six clock genes. You will propose continuous parameter targets that would subsequently be engineered. A gene may be associated with several model parameters, such as rates of transcription or promoter binding; the starter lists these associations in `DNA_GROUPS`. Changing any parameter in a group counts as editing that gene, while changing several parameters in the same group still counts as one gene edit. Modifying several genes is more expensive than modifying a single gene, so we also favour solutions that concentrate their edits in fewer genes.

The starter files list the editable parameters in `DNA_GROUPS`. You may choose their values independently, but all parameters **must remain positive**. Parameters outside these groups must retain their reporter values.

There is no unique solution to this problem. Since you are trying to restore the behaviour of an entire network, many combinations of modifications may achieve similar results. You should therefore aim to recover behaviour close to that of the wild-type system, rather than expecting a perfect restoration.

Your submission. Explore parameter changes in `challenge.py`. Save your final 45-parameter vector as `p_edit` in the separate file `challenge_submission.py`, starting from the reporter vector provided there. The autograder checks that the vector is valid, the edited model oscillates, the objective meets the threshold below, and no non-DNA parameters were changed.

The evaluation simulates the model at $\mathbf{p}_{\text{edit}}$ and measures the period and peak phases of the five selected proteins. It then computes the recovery error E according to the formula below.

Recovery error and objective. The recovery error E measures how closely the edited clock matches the wild-type measurements. The objective J adds gene-edit costs to the normalized recovery error. The recovery error E is computed using the following formulas

$$E = \frac{E_T^2 + E_\phi^2}{2} \qquad E_T = \frac{|T_{\text{edit}} - T_{\text{WT}}|}{T_{\text{WT}}} \qquad E_\phi = \sqrt{\frac{1}{5}\sum_{i=1}^{5} \delta_i^2}.$$

E_T is the period error which compares the wild-type and edited periods T_{WT} and T_{edit}.

E_ϕ is the phase error. The supplied code aligns the phases and calculates this error for you; you do not need to implement it. It is the root mean square of the five aligned phase differences.

Here, δ_i is the wrapped phase difference, in cycles, for the ith selected species after applying one common phase rotation. This rotation accounts for the arbitrary time origin of an autonomous cycle. The supplied routine finds it by averaging the phase differences as unit complex numbers.

To balance recovery against editing cost, we want to minimize: $J = \dfrac{E(\mathbf{p}_{\text{edit}})}{E(\mathbf{p}_{\text{rep}})} + [\text{gene edit cost}].$

Each gene that you are manipulating adds to the cost according to the table. For example if you modify PER and ROR (no matter how much) and leave all others unchanged, then the cost is $0.405 + 0.168 = 0.573$. Each edited gene is charged once, regardless of the size of its changes. Doing nothing gives $J = 1$. Improvement is $100(1 - J)\%$.

Completion threshold. Your vector must achieve at least 50% improvement, or $J \leq 0.50$. A vector that changes a non-DNA parameter is invalid, regardless of its score.

Gene	Cost
PER	0.405
CRY1	0.180
CRY2	0.156
ROR	0.168
REV	0.192
BMAL	0.204

The competition. Valid submissions are ranked by J. The lowest scores receive a MAT292 novelty mug and recognition in class.

Some suggestions on how to approach this. Your task of choosing an optimal intervention is an example of a **nonlinear optimization problem**. There are many numerical methods for solving such problems, and their development remains an active area of mathematical research. A common feature of these methods is their use of the **gradient**, which points in the direction of steepest increase of an objective function (and whose negative points in the direction of steepest decrease).

The finite-difference sensitivity analysis introduced in Q3 can provide gradient information to guide our choice of modifications. Ultimately, the effectiveness of an intervention must be assessed through numerical simulations, testing how different choices of $\Delta\mathbf{p}$ affect the system and identifying those that come closest to restoring wild-type behaviour. Rather than testing interventions blindly, we can use sensitivity information to identify promising candidates.

To do this, use the finite-difference methods from Q3 to estimate how changes in editable parameter directions affect the objective, then test promising updates by simulating the modified model. Complete the indicated starter function and use the supplied functions in `challenge.py` to evaluate your vectors. You need to implement the finite differences yourself. Keep in mind that local sensitivity estimates are only a guide, since the final modification are finite rather than infinitesimal.